

SKILL WEEK – 8

Session – 15

TASK 1: TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir -p ~/Week8_skill
cd ~/Week8_skill
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ nano session15_task1.c
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ gcc session15_task1.c -o session15_task1
./session15_task1
Session 15 - Combined Redirection
Enter command with combined redirection:
myshell> echo test > output.txt 2>&1

Redirection detected:
stdout -> output.txt
stderr -> stdout
Output file opened successfully.

Child process completed.
Child exit status: 0

Combined output file contents:
-----
This is standard output.
This is standard error.
Both streams are merged successfully.
-----
Output ordering verified.
Combined redirection completed successfully.
```

```
GNU nano 8.7.1 session15_task1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200

int main(void) {
    char input[MAX_INPUT];
    char filename[100];

    printf("Session 15 - Combined Redirection\n");
    printf("Enter command with combined redirection:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    /* Find output redirection > */
    char *redirect = strchr(input, '>');
    if (redirect == NULL) {
        printf("Error: Output redirection not found.\n");
        return 1;
    }

    /* Find combined redirection 2>&1 */
    char *merge = strstr(input, "2>&1");
    if (merge == NULL) {
        printf("Error: Use 2>&1 for combined redirection.\n");
        return 1;
    }

    /* Extract output filename between > and 2>&1 */
    char *file_start = redirect + 1;
    while (*file_start == ' ' || *file_start == '\t')
        file_start++;

    char *file_end = merge;
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session15_task1.c
GNU nano 8.7.1
char *file_end = merge;
while (file_end > file_start &&
      (*(file_end - 1) == ' ' || *(file_end - 1) == '\t'))
    file_end--;

int length = file_end - file_start;
if (length <= 0 || length >= 1000) {
    printf("Error: Invalid output filename.\n");
    return 1;
}

strncpy(filename, file_start, length);
filename[length] = '\0';

printf("\nRedirection detected:\n");
printf("stdout -> %s\n", filename);
printf("stderr -> stdout\n");

/* Open output file */
int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1) {
    perror("Error opening output file");
    return 1;
}

printf("Output file opened successfully.\n");

pid_t pid = fork();
if (pid < 0) {
    perror("fork failed");
    close(fd);
    return 1;
}

if (pid == 0) {
    /* Redirect stdout to file */
    if (dup2(fd, STDOUT_FILENO) == -1) {
        perror("dup2 stdout failed");
        close(fd);
        exit(EXIT_FAILURE);
    }
    /* Redirect stderr to stdout -> effectively to the file */
    if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1) {

```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session15_task1.c
GNU nano 8.7.1
        /* Redirect stderr to stdout -> effectively to the file */
        if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1) {
            perror("dup2 stderr failed");
            close(fd);
            exit(EXIT_FAILURE);
        }
        close(fd);

        printf("This is standard output.\n");
        fprintf(stderr, "This is standard error.\n");
        printf("Both streams are merged successfully.\n");
        exit(0);
    }
    close(fd);

    int status;
    waitpid(pid, &status, 0);

    printf("\nChild process completed.\n");
    if (WIFEXITED(status)) {
        printf("Child exit status: %d\n", WEXITSTATUS(status));
    }

    /* Verify output file */
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) {
        perror("Error reading output file");
        return 1;
    }

    char buffer[300];
    printf("\nCombined output file contents:\n");
    printf("-----\n");
    while (fgets(buffer, sizeof(buffer), fp) != NULL) {
        printf("%s", buffer);
    }
    printf("-----\n");
    fclose(fp);

    printf("Output ordering verified.\n");
    printf("Combined redirection completed successfully.\n");
    return 0;
}
```

TASK 2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```

shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ cat < input.txt | grep hello > out.txt
cmd1 | cmd2 > file
-bash: input.txt: No such file or directory
cmd1: command not found
cmd2: command not found
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ nano session15_task2.c
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ gcc session15_task2.c -o session15_task2
./session15_task2
Session 15 - Complex Pipeline with Redirections
Enter a complex command (pipes + redirections):
myshell> ls | grep c > out.txt

Syntax validation: SUCCESS
Execution order: Left to Right

--- Execution Plan ---
Command 1: ls
Command 2: grep c
> out.txt
-----

All commands executed successfully.
Complex pipeline execution completed.
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ cat out.txt
session15_task1.c
session15_task2.c
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ |

```

```

GNU nano 8.7.1 session15_task2.c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_CHDS 10
#define MAX_ARGS 20
#define MAX_LINE 500

typedef struct {
    char *argv[MAX_ARGS];
    int argc;
    char input_file[100]; /* for < */
    char output_file[100]; /* for > */
} Cmd;

/* Trim leading/trailing spaces from a string in-place */
static char *trim(char *s) {
    while (*s == ' ' || *s == '\t') s++;
    if (*s == '\0') return s;
    char *end = s + strlen(s) - 1;
    while (end > s && (*end == ' ' || *end == '\t')) *end-- = '\0';
    return s;
}

/* Parse one segment (no pipes). Extract redirections and args. */
static int parse_segment(char *seg, Cmd *cmd) {
    cmd->argc = 0;
    cmd->input_file[0] = '\0';
    cmd->output_file[0] = '\0';

    char *saveptr;
    char *tok = strtok_r(seg, " \t", &saveptr);

    while (tok != NULL) {
        if (strcmp(tok, "<") == 0) {
            char *f = strtok_r(NULL, " \t", &saveptr);
            if (f == NULL) return -1;
            strncpy(cmd->input_file, f, sizeof(cmd->input_file) - 1);

```

```

shreyaj@LAPTOP-FT6VVM4D: x + v session15_task2.c
GNU nano 8.7.1
    strncpy(cmd->input_file, f, sizeof(cmd->input_file) - 1);
    cmd->input_file[sizeof(cmd->input_file) - 1] = '\0';
} else if (strcmp(tok, ">") == 0) {
    char *f = strtok_r(NULL, " \t", &saveptr);
    if (f == NULL) return -1;
    strncpy(cmd->output_file, f, sizeof(cmd->output_file) - 1);
    cmd->output_file[sizeof(cmd->output_file) - 1] = '\0';
} else {
    if (cmd->argc < MAX_ARGS - 1) {
        cmd->argv[cmd->argc++] = tok;
    }
    tok = strtok_r(NULL, " \t", &saveptr);
}
cmd->argv[cmd->argc] = NULL;
return cmd->argc > 0 ? 0 : -1;
}

int main(void) {
    char line[MAX_LINE];
    Cmd cmds[MAX_CMDS];

    printf("Session 15 - Complex Pipeline with Redirections\n");
    printf("Enter a complex command (pipes + redirections):\n");
    printf("myshell> ");

    if (fgets(line, sizeof(line), stdin) == NULL)
        return 1;

    line[strcspn(line, "\n")] = '\0';

    if (strlen(line) == 0) {
        printf("Empty command.\n");
        return 0;
    }

    /* Split by pipe */
    int count = 0;
    char *saveptr;
    char *part = strtok_r(line, "|", &saveptr);

    while (part != NULL && count < MAX_CMDS) {
        char *trimmed = trim(part);

```

```

shreyaj@LAPTOP-FT6VVM4D: x + v session15_task2.c
GNU nano 8.7.1
        char *trimmed = trim(part);
        if (*trimmed == '\0') {
            printf("Error: Empty command in pipeline.\n");
            return 1;
        }
        if (parse_segment(trimmed, &cmds[count]) != 0) {
            printf("Error: Invalid command segment.\n");
            return 1;
        }
        count++;
        part = strtok_r(NULL, "|", &saveptr);
    }

    printf("\nSyntax validation: SUCCESS\n");
    printf("Execution order: Left to Right\n");

    /* Print execution plan */
    printf("\n--- Execution Plan ---\n");
    for (int i = 0; i < count; i++) {
        printf("Command %d: ", i + 1);
        for (int j = 0; j < cmds[i].argc; j++) {
            printf("%s ", cmds[i].argv[j]);
        }
        printf("\n");
        if (cmds[i].input_file[0])
            printf("  < %s\n", cmds[i].input_file);
        if (cmds[i].output_file[0])
            printf("  > %s\n", cmds[i].output_file);
    }
    printf("-----\n");

    /* Create pipes */
    int pipes[MAX_CMDS - 1][2];
    for (int i = 0; i < count - 1; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe failed");
            return 1;
        }
    }

    pid_t pids[MAX_CMDS];

    for (int i = 0; i < count; i++) {

```

```
shreyaj@LAPTOP-FT6VVM4D: ~$ nano session15_task2.c
GNU nano 8.7.1 session15_task2.c
for (int i = 0; i < count; i++) {
    pids[i] = fork();
    if (pids[i] < 0) {
        perror("fork failed");
        return 1;
    }

    if (pids[i] == 0) {
        /* stdin from previous pipe */
        if (i > 0) {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }

        /* stdout to next pipe */
        if (i < count - 1) {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        /* Input redirection overrides pipe-in */
        if (cmds[i].input_file[0]) {
            int fd = open(cmds[i].input_file, O_RDONLY);
            if (fd == -1) {
                perror("Input file error");
                exit(EXIT_FAILURE);
            }
            dup2(fd, STDIN_FILENO);
            close(fd);
        }

        /* Output redirection overrides pipe-out */
        if (cmds[i].output_file[0]) {
            int fd = open(cmds[i].output_file,
                          O_WRONLY | O_CREAT | O_TRUNC, 0644);
            if (fd == -1) {
                perror("Output file error");
                exit(EXIT_FAILURE);
            }
            dup2(fd, STDOUT_FILENO);
            close(fd);
        }

        /* Close all pipe descriptors in child */
        for (int j = 0; j < count - 1; j++) {
            close(pipes[j][0]);
        }

        /* Output redirection overrides pipe-out */
        if (cmds[i].output_file[0]) {
            int fd = open(cmds[i].output_file,
                          O_WRONLY | O_CREAT | O_TRUNC, 0644);
            if (fd == -1) {
                perror("Output file error");
                exit(EXIT_FAILURE);
            }
            dup2(fd, STDOUT_FILENO);
            close(fd);
        }

        /* Close all pipe descriptors in child */
        for (int j = 0; j < count - 1; j++) {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        execvp(cmds[i].argv[0], cmds[i].argv);
        perror("Execution failed");
        exit(EXIT_FAILURE);
    }
}

/* Parent closes all pipe descriptors */
for (int i = 0; i < count - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}

for (int i = 0; i < count; i++) {
    waitpid(pids[i], NULL, 0);
}

printf("\nAll commands executed successfully.\n");
printf("Complex pipeline execution completed.\n");
return 0;
}
```

Session – 16

TASK 1: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```

shreyaj@LAPTOP-FT6VVM4D: ~$ cd ~/Week8_skill
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ nano session16_task1.c
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ gcc session16_task1.c -o session16_task1
./session16_task1
Session 16 - Background Jobs
Commands: run, jobs, exit
myshell> run
Background job started: [1] PID 765
Prompt returned immediately.
myshell>
Background process started. PID: 765

Background process completed. PID: 765
jobs

Background process completed. PID: 765

Background Jobs:
-----
Job ID  PID      Status      Command
-----
[1]     765      Completed   sleep 5
-----
myshell> exit

Waiting for background jobs...
All background jobs completed.
Exiting shell...
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ |

```

```

GNU nano 8.7.1 session16_task1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct {
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

/* Add a new job */
void add_job(pid_t pid, const char *cmd) {
    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    strncpy(jobs[job_count].command, cmd, sizeof(jobs[job_count].command) - 1);
    jobs[job_count].command[sizeof(jobs[job_count].command) - 1] = '\0';
    strcpy(jobs[job_count].status, "Running");
    job_count++;
}

/* Check status of background jobs without blocking */
void monitor_jobs(void) {
    for (int i = 0; i < job_count; i++) {
        if (strcmp(jobs[i].status, "Running") != 0)
            continue;

        int status;
        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);
        if (result > 0) {
            if (WIFEXITED(status)) {

```

```

shreyaj@LAPTOP-FT6VVM4D: x + v session16_task1.c
GNU nano 8.7.1
    if (WIFEXITED(status)) {
        strcpy(jobs[i].status, "Completed");
        printf("\nBackground process completed. PID: %d\n", jobs[i].pid);
    } else {
        strcpy(jobs[i].status, "Terminated");
        printf("\nBackground process terminated. PID: %d\n", jobs[i].pid);
    }
}
}
}

/* Display background jobs */
void show_jobs(void) {
    monitor_jobs();
    printf("\nBackground Jobs:\n");
    printf("-----\n");
    printf("Job ID\tPID\tStatus\tCommand\n");
    printf("-----\n");

    if (job_count == 0) {
        printf("No background jobs.\n");
    } else {
        for (int i = 0; i < job_count; i++) {
            printf("[%d]\t%d\t%-10s\t%s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].status,
                jobs[i].command);
        }
    }
    printf("-----\n");
}

int main(void) {
    char input[100];

    printf("Session 16 - Background Jobs\n");
    printf("Commands: run, jobs, exit\n");

    while (1) {
        /* Check background jobs without blocking */
        monitor_jobs();

```

```

shreyaj@LAPTOP-FT6VVM4D: x + v session16_task1.c
GNU nano 8.7.1
    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "run") == 0) {
        pid_t pid = fork();
        if (pid < 0) {
            perror("fork failed");
        } else if (pid == 0) {
            /* Child: simulate background task */
            printf("\nBackground process started. PID: %d\n", getpid());
            sleep(5);
            printf("\nBackground process completed. PID: %d\n", getpid());
            exit(0);
        } else {
            add_job(pid, "sleep 5");
            printf("Background job started: [%d] PID %d\n",
                job_count, pid);
            printf("Prompt returned immediately.\n");
        }
    } else if (strcmp(input, "jobs") == 0) {
        show_jobs();
    } else if (strcmp(input, "exit") == 0) {
        printf("\nWaiting for background jobs...\n");
        for (int i = 0; i < job_count; i++) {
            if (strcmp(jobs[i].status, "Running") == 0) {
                waitpid(jobs[i].pid, NULL, 0);
                strcpy(jobs[i].status, "Completed");
            }
        }
        printf("All background jobs completed.\n");
        printf("Exiting shell...\n");
        break;
    } else if (strlen(input) == 0) {
        continue;
    } else {
        printf("Invalid command: %s\n", input);
        printf("Use: run, jobs, or exit\n");
    }
}

```

TASK 2: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```

shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ nano session16_task2.c
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$ gcc session16_task2.c -o session16_task2
./session16_task2
Session 16 - Job Table Management
Commands: run, jobs, update, remove, validate, exit
myshell> run
Job created: [1] PID: 853 PGID: 853
myshell>
Child process started.
PID: 853
PGID: 853

myshell> jobs

Job Table:
-----
Job ID  PID    PGID   State    Command
-----
[1]     853     853   Running   sleep 8
-----

myshell>
Child process completed.

myshell> update
Job states updated successfully.

Job Table:
-----
Job ID  PID    PGID   State    Command
-----
[1]     853     853   Completed sleep 8
-----

myshell> remove
Removing Job [1] from job table.
Job table cleanup completed.
myshell> validate
All job records are valid.
myshell> exit

Cleaning up jobs...
All resources cleaned up.
Exiting...
shreyaj@LAPTOP-FT6VVM4D:~/Week8_skill$

```

```

GNU nano 8.7.1 session16_task2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 10

typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    char state[20];
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, pid_t pgid, const char *command) {
    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }
    job_table[job_count].job_id = job_count + 1;
    job_table[job_count].pid = pid;
    job_table[job_count].pgid = pgid;
    strncpy(job_table[job_count].command, command,
            sizeof(job_table[job_count].command) - 1);
    job_table[job_count].command[sizeof(job_table[job_count].command) - 1] = '\0';
    strcpy(job_table[job_count].state, "Running");
    job_count++;
}

void update_jobs(void) {
    for (int i = 0; i < job_count; i++) {
        if (strcmp(job_table[i].state, "Running") != 0)
            continue;

        int status;
        pid_t result = waitpid(job_table[i].pid, &status, WNOHANG);
    }
}

```



```
shreyaj@LAPTOP-FT6VVM4D: x + v session16_task2.c
GNU nano 8.7.1
    if (result > 0) {
        if (WIFEXITED(status))
            strcpy(job_table[i].state, "Completed");
        else
            strcpy(job_table[i].state, "Terminated");
    }
}

void remove_completed_jobs(void) {
    int w = 0;
    for (int i = 0; i < job_count; i++) {
        if (strcmp(job_table[i].state, "Running") == 0) {
            job_table[w++] = job_table[i];
        } else {
            printf("Removing Job [%d] from job table.\n", job_table[i].job_id);
        }
    }
    job_count = w;
    /* Renumber */
    for (int i = 0; i < job_count; i++)
        job_table[i].job_id = i + 1;
}

void show_jobs(void) {
    update_jobs();
    printf("\nJob Table:\n");
    printf("-----\n");
    printf("Job ID\tPID\tPGID\tState\tCommand\n");
    printf("-----\n");

    if (job_count == 0) {
        printf("No jobs in table.\n");
    } else {
        for (int i = 0; i < job_count; i++) {
            printf("[%d]\t%d\t%d\t%-10s\t%s\n",
                job_table[i].job_id,
                job_table[i].pid,
                job_table[i].pgid,
                job_table[i].state,
                job_table[i].command);
        }
    }
}

}
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session16_task2.c
GNU nano 8.7.1
    }
    printf("-----\n");
}

void validate_jobs(void) {
    int ok = 1;
    for (int i = 0; i < job_count; i++) {
        if (job_table[i].pid <= 0) ok = 0;
        if (job_table[i].pgid <= 0) ok = 0;
        if (strlen(job_table[i].state) == 0) ok = 0;
    }
    if (ok)
        printf("All job records are valid.\n");
    else
        printf("Some job records are invalid.\n");
}

int main(void) {
    char input[100];

    printf("Session 16 - Job Table Management\n");
    printf("Commands: run, jobs, update, remove, validate, exit\n");

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "run") == 0) {
            pid_t pid = fork();
            if (pid < 0) {
                perror("fork failed");
            } else if (pid == 0) {
                /* New process group */
                setpgid(0, 0);
                printf("\nChild process started.\n");
                printf("PID: %d\n", getpid());
                printf("PGID: %d\n", getpgid());
                sleep(8);
            }
        }
    }
}
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session16_task2.c
GNU nano 8.7.1
sleep(0);
printf("\nChild process completed.\n");
exit(0);
} else {
    setpgid(pid, pid);
    add_job(pid, pid, "sleep 8");
    printf("Job created: [%d] PID: %d PGID: %d\n",
        job_count, pid, pid);
}
} else if (strcmp(input, "jobs") == 0) {
    show_jobs();
} else if (strcmp(input, "update") == 0) {
    update_jobs();
    printf("Job states updated successfully.\n");
    show_jobs();
} else if (strcmp(input, "remove") == 0) {
    update_jobs();
    remove_completed_jobs();
    printf("Job table cleanup completed.\n");
} else if (strcmp(input, "validate") == 0) {
    validate_jobs();
} else if (strcmp(input, "exit") == 0) {
    printf("\nCleaning up jobs...\n");
    for (int i = 0; i < job_count; i++) {
        if (strcmp(job_table[i].state, "Running") == 0) {
            kill(job_table[i].pid, SIGTERM);
            waitpid(job_table[i].pid, NULL, 0);
        }
    }
    printf("All resources cleaned up.\n");
    printf("Exiting...\n");
    break;
} else if (strlen(input) == 0) {
    continue;
} else {
    printf("Invalid command.\n");
    printf("Use: run, jobs, update, remove, validate, exit\n");
}
}
return 0;
}
```